

Type Checking

1. Type Checking

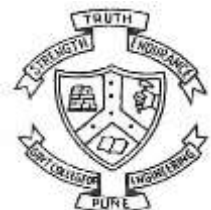
- Need of type checking?



Type Checking

1. Type Checking

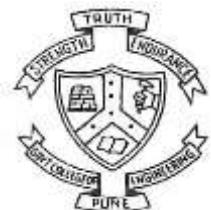
- Need of type checking?
- **Type checking is the activity of ensuring that the operands of an operator are of compatible types.**



Type Checking

1. Type Checking

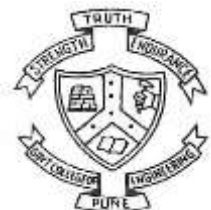
- Need of type checking?
- Type checking is the activity of ensuring that the operands of an operator are of compatible types.
- **What is compatible type?**



Type Checking

1. Type Checking

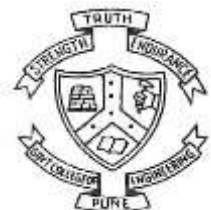
- Need of type checking?
- Type checking is the activity of ensuring that the operands of an operator are of compatible types.
- What is compatible type?
- **A compatible type is one that either is legal for the operator or is allowed under language rules to be implicitly converted by compiler-generated code (or the interpreter) to a legal type.**



Type Checking

1. Type Checking

- Need of type checking?
- Type checking is the activity of ensuring that the operands of an operator are of compatible types.
- What is compatible type?
- A compatible type is one that either is legal for the operator or is allowed under language rules to be implicitly converted by compiler-generated code (or the interpreter) to a legal type.
- **This automatic conversion is called a coercion.**
- **Example in java: `int(Coerced to float) + float = float`.**



Type Checking

2. Type Error

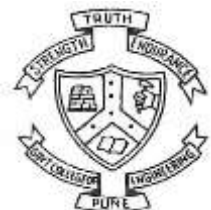
- Incompatible type will lead to type error.
- **In C if we are passing int value to a function which is expecting float value there a type error occurs.**



Strong Typing

1. Strong Type

- **Strongly typed languages enforce rules that help ensure that operations are performed on compatible data types. This reduces the chances of runtime errors caused by type mismatches.**



Strong Typing

1. Strong Type

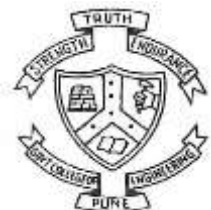
- Strongly typed languages enforce rules that help ensure that operations are performed on compatible data types. This reduces the chances of runtime errors caused by type mismatches.
- **If you want to use different types together (e.g., combining an integer with a string), you must explicitly convert one type to another.**



Strong Typing

1. Strong Type

- Strongly typed languages enforce rules that help ensure that operations are performed on compatible data types. This reduces the chances of runtime errors caused by type mismatches.
- If you want to use different types together (e.g., combining an integer with a string), you must explicitly convert one type to another.
- **Many strongly typed languages catch type errors at compile time, which can prevent bugs from making it into production.**



Strong Typing

1. Strong Type

- Strongly typed languages enforce rules that help ensure that operations are performed on compatible data types. This reduces the chances of runtime errors caused by type mismatches.
- If you want to use different types together (e.g., combining an integer with a string), you must explicitly convert one type to another.
- Many strongly typed languages catch type errors at compile time, which can prevent bugs from making it into production.

Example: `int i =5;`

`string j=“Hello”;`

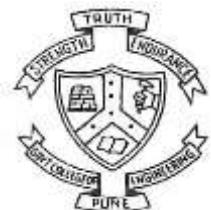
`string result=i+j //Compile time error.`



Strong Typing

1. Weak Type

- In languages like JavaScript or PHP, types can be implicitly converted. This can lead to unexpected results.

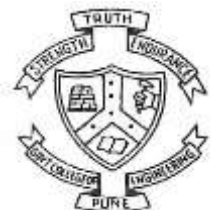


Strong Typing

1. Weak Type

- In languages like JavaScript or PHP, types can be implicitly converted. This can lead to unexpected results.

Example: `var number = 5;`
`var text = "Hello";`
`var result = number + text; // Result: "5Hello"`



Type Conversion

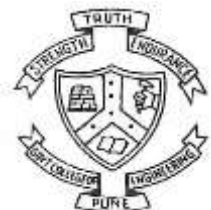
1. What is Type conversion.



Type Conversion

1. What is Type conversion.

- **Type conversion (or type casting) is the process of converting a value from one data type to another. It's often necessary when you want to perform operations on values of different types or when you want to store a value in a variable of a different type.**



Type Conversion

1. What is Type conversion.

- Type conversion (or type casting) is the process of converting a value from one data type to another. It's often necessary when you want to perform operations on values of different types or when you want to store a value in a variable of a different type.

1.1 Types of Type Conversion

- **Implicit Conversion (Coercion):**

This is automatically handled by the programming language. For example, when a smaller data type (like an integer) is used in a context that requires a larger data type (like a float), the language automatically converts the integer to a float.



Type Conversion

1.1 Types of Type Conversion

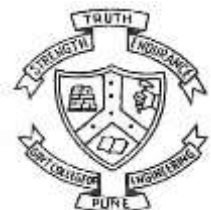
- **Implicit Conversion (Coercion):**

This is automatically handled by the programming language. For example, when a smaller data type (like an integer) is used in a context that requires a larger data type (like a float), the language automatically converts the integer to a float.

Example:

```
int_value = 5
```

```
float_value = int_value + 2.5  # int is implicitly converted to float
```



Type Conversion

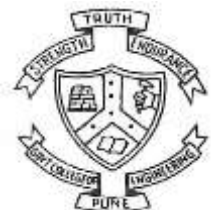
1.1 Types of Type Conversion

- **Explicit Conversion (Coercion):**

This requires the programmer to manually convert the type using a casting function or method. This approach is necessary when automatic conversion might lead to data loss or unintended results.

Example:

```
double doubleValue = 9.7;
int intValue = (int) doubleValue;
// Explicitly casting double to int
// intValue will be 9, the decimal part is discarded
```

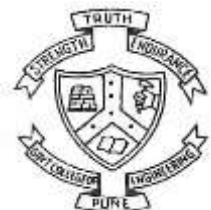


Type Conversion

1.1 Types of Type Conversion

- **Narrowing Conversion (Coercion):**

A narrowing conversion converts a value to a type that cannot store even approximations of all of the values of the original type.



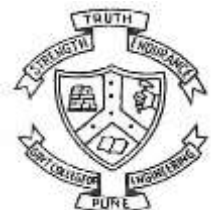
Type Conversion

1.1 Types of Type Conversion

Narrowing Conversion (Coercion):

A narrowing conversion converts a value to a type that cannot store even approximations of all of the values of the original type.

For example, converting a double to a float in Java is a narrowing conversion, because the range of double is much larger than that of float.

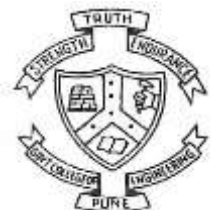


Type Conversion

1.1 Types of Type Conversion

Narrowing Conversion (Coercion):

1. A narrowing conversion converts a value to a type that cannot store even approximations of all of the values of the original type.
2. For example, converting a double to a float in Java is a narrowing conversion, because the range of double is much larger than that of float.
3. **Narrowing conversions are not always safe because there is a possibility of data or precision loss.**

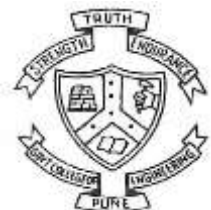


Type Conversion

1.1 Types of Type Conversion

Widening Conversion (Coercion):

1. A widening conversion converts a value to a type that can include at least approximations of all of the values of the original type.

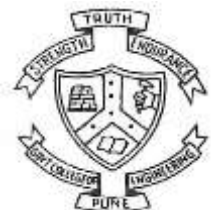


Type Conversion

1.1 Types of Type Conversion

Widening Conversion (Coercion):

1. A widening conversion converts a value to a type that can include at least approximations of all of the values of the original type.
2. **For example, converting an int to a float in Java is a widening conversion.**

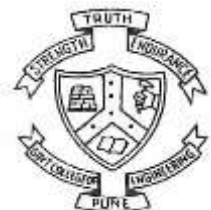


Type Conversion

1.1 Types of Type Conversion

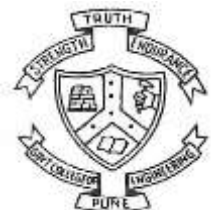
Widening Conversion (Coercion):

1. A widening conversion converts a value to a type that can include at least approximations of all of the values of the original type.
2. For example, converting an int to a float in Java is a widening conversion.
3. **Widening conversions are nearly always safe**



Short Circuit Evaluation

- A short-circuit evaluation of an expression is one in which the result is determined without evaluating all of the operands and/or operators.



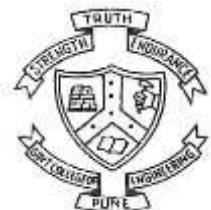
Short Circuit Evaluation

- A short-circuit evaluation of an expression is one in which the result is determined without evaluating all of the operands and/or operators.
- **Why we need short circuit evaluation?**



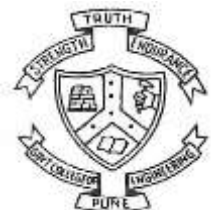
Short Circuit Evaluation

- A short-circuit evaluation of an expression is one in which the result is determined without evaluating all of the operands and/or operators.
- Why we need short circuit evaluation?
- **Ex: $(13 * a) * (b / 13 - 1)$.**



Short Circuit Evaluation

- A short-circuit evaluation of an expression is one in which the result is determined without evaluating all of the operands and/or operators.
- Why we need short circuit evaluation?
- Ex: $(13 * a) * (b / 13 - 1)$.
- **If a is 0 then no need to evaluate $(b / 13 - 1)$ because answer of above expression is 0.**



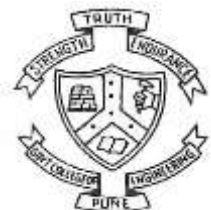
Short Circuit Evaluation

- A short-circuit evaluation of an expression is one in which the result is determined without evaluating all of the operands and/or operators.
- Why we need short circuit evaluation?
- Ex: $(13 * a) * (b / 13 - 1)$.
- If a is 0 then no need to evaluate $(b / 13 - 1)$ because answer of above expression is 0.
- **Its unnecessary increasing the computation cost of the program hence we need short circuit evaluation.**



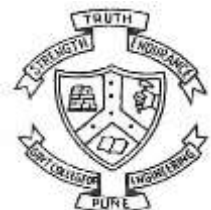
Short Circuit Evaluation

- A short-circuit evaluation of an expression is one in which the result is determined without evaluating all of the operands and/or operators.
- Why we need short circuit evaluation?
- Ex: $(13 * a) * (b / 13 - 1)$.
- If a is 0 then no need to evaluate $(b / 13 - 1)$ because answer of above expression is 0.
- Its unnecessary increasing the computation cost of the program hence we need short circuit evaluation.
- **Short-circuit evaluation is a programming technique used in conditional expressions (like logical AND and OR) where the evaluation of an expression stops as soon as the result is determined.**



Short Circuit Evaluation

- Short-circuit evaluation is a programming technique used in conditional expressions (like logical AND and OR) where the evaluation of an expression stops as soon as the result is determined.

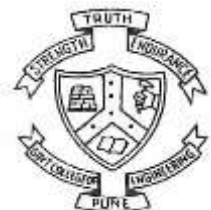


Short Circuit Evaluation

- Short-circuit evaluation is a programming technique used in conditional expressions (like logical AND and OR) where the evaluation of an expression stops as soon as the result is determined.
- **Example:**
- **In an expression like `A && B`, if `A` evaluates to false, then `B` is not evaluated because the entire expression can't be true if the first operand is false.**

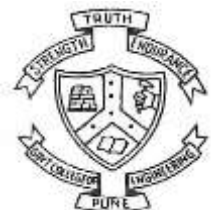
let `a = false`;

let `b = (a && x+1)`; // `X+1` is not execute.



Short Circuit Evaluation

- Problems of short circuit evaluation.
- If program correctness depends on the side effect, short-circuit evaluation can result in a serious error.
- Example: $(a > b) \parallel ((b++) / 3)$.
- b is changes only if $a \leq b$ If the programmer assumed b would be changed every time this expression is evaluated during execution (and the program's correctness depends on it), the program will fail.



Thank You



Department of Computer Engineering and Information Technology
College of Engineering Pune (COEP)
Forerunners in Technical Education